Member 1 – Document Processing

PDF → Clean Text → Structured Information

Objective: Take a PDF/document, extract its content, clean the text, divide it into meaningful chunks, identify important information using NLP, and produce structured JSON data for the next stage.

1. What You Need to Do

• Read the PDF using PyMuPDF.

• Extract text, images (if required), and document information.

• Clean the extracted text by removing extra spaces, unwanted symbols, headers, footers, and noise.

• Split the text into meaningful chunks.

• Use NLP to identify important information such as people, dates, organizations, and locations.

• Save the final information as clean, structured JSON data.

2. Technologies Used

Python – main programming language.
PyMuPDF (fitz) – reads PDFs and extracts text.
spaCy – performs NLP and named-entity recognition.
Regular Expressions (re) – cleans unwanted text.
JSON – stores the final structured information.

3. Installation

```
pip install pymupdf spacy
python -m spacy download en_core_web_sm
```

4. Complete Python Code

```
import fitz          # PyMuPDF
import re
import spacy
import json

# Load NLP model
nlp = spacy.load("en_core_web_sm")


# ------------------------------------------------
# STEP 1: Extract text from PDF
# ------------------------------------------------

def extract_text_from_pdf(pdf_path):
    document = fitz.open(pdf_path)

    full_text = ""

    for page in document:
        text = page.get_text()
        full_text += text + "\n"

    document.close()

    return full_text


# ------------------------------------------------
# STEP 2: Clean the extracted text
# ------------------------------------------------
```

```
def clean_text(text):

    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text)

    # Remove unwanted special characters
    text = re.sub(r'[^\w\s.,!?;:()@/-]', '', text)

    # Remove leading/trailing spaces
    text = text.strip()

    return text


# ------------------------------------------------
# STEP 3: Split text into meaningful chunks
# ------------------------------------------------

def split_into_chunks(text, chunk_size=500):

    words = text.split()
    chunks = []

    for i in range(0, len(words), chunk_size):
        chunk = " ".join(words[i:i + chunk_size])
        chunks.append(chunk)

    return chunks


# ------------------------------------------------
# STEP 4: Extract important information using NLP
# ------------------------------------------------

def extract_entities(text):

    doc = nlp(text)
    entities = []

    for ent in doc.ents:
        entities.append({
            "text": ent.text,
            "type": ent.label_
        })

    return entities


# ------------------------------------------------
# STEP 5: Create structured information
# ------------------------------------------------

def create_structured_data(text):

    entities = extract_entities(text)

    people = []
    dates = []
    organizations = []
    locations = []

    for entity in entities:

        if entity["type"] == "PERSON":
            people.append(entity["text"])

        elif entity["type"] in ["DATE", "TIME"]:
            dates.append(entity["text"])

        elif entity["type"] == "ORG":
            organizations.append(entity["text"])

        elif entity["type"] == "GPE":
            locations.append(entity["text"])

    structured_data = {
        "people": list(set(people)),
        "dates": list(set(dates)),
        "organizations": list(set(organizations)),
        "locations": list(set(locations)),
        "entities": entities
    }
```

```
    return structured_data


# ------------------------------------------------
# MAIN PROGRAM
# ------------------------------------------------

pdf_path = "sample.pdf"

# 1. Extract
raw_text = extract_text_from_pdf(pdf_path)

print("\n========== RAW TEXT ==========")
print(raw_text)


# 2. Clean
cleaned_text = clean_text(raw_text)

print("\n========== CLEAN TEXT ==========")
print(cleaned_text)


# 3. Split into chunks
chunks = split_into_chunks(cleaned_text)

print("\n========== TEXT CHUNKS ==========")

for i, chunk in enumerate(chunks):
    print(f"\nChunk {i + 1}:")
    print(chunk)


# 4. Extract important information
structured_data = create_structured_data(cleaned_text)

print("\n========== STRUCTURED DATA ==========")
print(json.dumps(structured_data, indent=4))


# ------------------------------------------------
# SAVE OUTPUT AS JSON
# ------------------------------------------------

with open("structured_data.json", "w", encoding="utf-8") as file:
    json.dump(structured_data, file, indent=4, ensure_ascii=False)

print("\nData saved successfully to structured_data.json")
```

5. Project Workflow

```
PDF
 ↓
PyMuPDF
 ↓
Extract Text
 ↓
Clean Text
 ↓
Split into Chunks
 ↓
NLP / spaCy
 ↓
Extract Important Information
 ↓
Structured JSON
 ↓
Next Stage
```

6. Example

Suppose the PDF contains: “A meeting was held on 20 September 2026. Rahul and Priya attended the meeting at ABC Technologies in Delhi.”

The program can produce:

```
{
    "people": [
        "Rahul",
        "Priya"
    ],
    "dates": [
        "20 September 2026"
    ],
    "organizations": [
        "ABC Technologies"
    ],
    "locations": [
        "Delhi"
    ]
}
```

7. 60-Second Explanation for Presentation

“My role is Document Processing. I take the input PDF and use PyMuPDF to extract its text. Then I clean the text by removing unwanted spaces, symbols and noise. I divide the text into meaningful chunks and use NLP with spaCy to identify important entities such as people, dates, organizations and locations. Finally, I convert this information into structured JSON data, which is passed to the next stage of our project.”

8. Important Note

The file name in the code is **sample.pdf**. Put your actual PDF in the same folder as the Python program, or change the pdf_path variable to the correct file path.